

Functional Test Plan for InvestMate

David Casente, Rembrandt van der Ploeg, Mike O'Brien

November 6, 2019

Contents

Revision History	3
1 Introduction	4
2 Project and Scope	4
3 Test Design Specifications	4
4 Test Specifications	5
4.1 Importing Portfolio	5
4.2 Editing Portfolio	5
4.3 View Overall Portfolio	6
4.4 View Specific Stock	6
4.5 View Projection	6
4.6 Stock of the Week	6
4.7 Analyst Stock of the Week	7
4.8 Search for Stocks	7
4.9 Metric Calculation and Explanation	7
4.10 Buy Stock	8
4.11 Sell Stock	8
4.12 Submit Question for Analyst	9
5 Functional Tests	9
5.1 Analyst Class Tests	9
5.2 Graphics Class Tests	9
5.3 Individual Stock Assessment Class Tests	10
5.4 Owned Stock Class Tests	10
5.5 Portfolio Class Tests	11
5.6 Project Learning Tool Class Tests	11
5.7 Question Class Tests	12
5.8 Stock Class Tests	12
5.9 Stock Analysis Class Tests	13
5.10 Stock Retrieval Class Tests	13
5.11 Stock Search Recommender Class Tests	13
5.12 User Class Tests	14
6 Defect Responsibility and Resolution	14
7 Exit Criteria	15

Revision History

Name	Date	Reason For Change	Version
Mike, David, and Remy	11/1/19	Initial Draft	1.0 draft 1
Mike, David, and Remy	11/4/19	Baseline changes after inspection	1.1 draft 2
Mike, David, and Remy	11/6/19	Final Initial Draft Revisions	1.2 draft 3

1 Introduction

This document serves as an outline of the goals for testing of the InvestMate application. It will entail all the tests for the methods within the application and how we plan on implementing them. Functional testing areas include stock data retrieval, stock/portfolio analysis and professional analyst response. The objectives of this document are to outline tests and create detailed guidelines for function testing plan. The other objective is to confirm that each test case verifies the software and fulfills the requirements that we talked about in our SRS document.

2 Project and Scope

This document will present the goals which our group will accomplish so that the InvestMate application can be fully tested. It will contain brief descriptions of how each of the use cases will be tested. We will mainly test the InvestMate application using Junit tests, but we will also use some special testing for the data retrieval. For specific tests, reference the Test Specifications section of this document. We have made it so all the user can input into the program is through pre selected data, so that we can control the range of tests that we run on the program. Therefore, we will be testing a limited but comprehensive sample size. After completing the code for each of the sections of our program we will begin to test it. Once we have finished testing that one section of our program, we will continue to implement another section of our project.

3 Test Design Specifications

In this document, we will have specific tests for each of our different classes. Each test will be simulated with a user input and then go through steps to get the expected output of the program. If all the steps of the program have the correct output for the input, then the test will count as being passed. Any test that fails somewhere along the way to the final step of testing will be considered as a failed test.

4 Test Specifications

4.1 Importing Portfolio

1. user signs up on the login page.
2. The application returns the Portfolio Onboarding page with input spaces for stocks, with parameters for stock name, amount, and date purchased.
3. The user completes the input page.

If the user does not fill out a section for the stock, or inputs an invalid value, the application will prevent the user from continuing until the mistake is fixed
4. The application takes the user's inputs as arrays of string for stock name, int for amount and string for date purchased. The application creates OwnedStock objects for each stock inputted, uses them to create a Portfolio object and creates a User object for the user, with a variable pointing to the portfolio object.
5. The application returns the main page, with the Portfolio object displayed.

4.2 Editing Portfolio

1. The user clicks the "Edit Portfolio" button which is located on the Main page.
2. The application then returns the Edit Portfolio page, which displays all stocks currently in a user portfolio, with a delete button as well as a button at the bottom of the list for adding stocks.

if the user clicks on the button for adding stocks, an empty stock section will appear for the user to fill out.

if the user clicks on the delete button, then that stock will be removed from the portfolio of the user.
3. The user can then edit the portfolio either removing or adding stocks and submit the changes.
4. The application takes the user's inputs as arrays of string for stock name, int for amount and string for date purchased and updates the user's portfolio object.

5. The application returns the Main page.

4.3 View Overall Portfolio

1. The user clicks the “Return Home” button on any other page in the application.
2. The application then returns the home page and displays the user’s portfolio.

4.4 View Specific Stock

1. The user clicks on a “more information” button designated to a specific stock on either the Main Page, Portfolio Assessment Page, or Stock Recommender Page.
2. The application then takes the input OwnedStock object, gets the Stock object associated with it and returns the Specific Stock Page for the input stock.

4.5 View Projection

1. The user clicks on a “View Projection Specifics” button designated to a specific stock the Main Page, Portfolio Assessment Page, or Stock Recommender Page.
2. The application then takes the Stock object as an input, returns the Projection Learning tool page
3. the application then creates a ProjectLearningTool object from the input Stock Object and displays the Projection of the stock and the explanation of it.

4.6 Stock of the Week

1. The user clicks on the stock search recommender tab on the Main Page.
2. The application then returns the stock search recommender page, with filter fields and a search button.
3. The user searches for a stock and does not enter any information in the filter fields

4. The application displays at the top the application's pick of the week, based on examination of all stocks in the database.

4.7 Analyst Stock of the Week

1. The use case begins with the analyst submitting their stock pick of the week to the application on the analyst page
2. The application will take the Stock object as an input and will update the analyst stock pick variable in the Stock Search Class
3. The user can then click on the stock search recommender tab on the Main Page
4. The user searches for a stock and checks the analyst field.
5. The application then returns the analyst pick of the week

4.8 Search for Stocks

1. The user clicks on the stock search recommender tab on the Main page.
2. The application returns the stock recommender page with filter fields and a search button.
3. The user fills out the filter fields and submit a search.
4. The application will take the input filters and determines using an algorithm the stock(s) that are recommendable or none at all and will display up to five stocks on the screen for the user.

4.9 Metric Calculation and Explanation

1. The user clicks on the individual stock assessment tab and then selecting a specific stock in the input filter fields and clicks the "Learn Metric Calculation" button or if the user clicks on the button "Learn Metric Calculation" on the specific stock page.
2. The application will take the Stock Object as an input and create a ProjectLearningTool Object for the stock.
3. The application will display the detailed calculation for the important investing metrics.

4.10 Buy Stock

1. The user clicks on a “more information” button designated to a specific stock on either the Main Page, Portfolio Assessment Page, or Stock Recommender Page.
2. The application then returns the Specific Stock Page, taking the input stock as an input, with a Buy Stock button at the bottom
3. The user can click on the button.
4. The application will take in the input stock and display a stock field, with the stock name field filled in and other fields blank.
5. The user will complete the fields and press the add to portfolio button
6. The application will use the input fields of stock name, amount and today’s date. The application will create an OwnedStock object for the input stock and add it to the user’s portfolio

4.11 Sell Stock

1. The user clicks on a “more information” button designated to a specific stock that they own on either the Main Page, Portfolio Assessment Page, or Stock Recommender Page.
2. The application then returns the Specific Stock Page, using the input stock, with a Sell Stock button at the bottom if they own the stock in their portfolio.
3. The user can click on the button sell stock.
4. The application take in the input stock object and display an input field for number of stocks, up to the amount the user owns.
5. The user will complete the fields and submit.
6. The application will use the input field and stock and will remove the amount of stock or the specific stock if they remove all, from the user’s portfolio.

4.12 Submit Question for Analyst

1. The user clicks on a “Ask an Analyst” button on either the specific stock page or the Projection Learning Tool
2. The application then displays the ask an analyst page with the section for submitting questions at the top, which includes input fields for type of question, description, etc.
3. The user then fills out the fields and submits their question
4. The application will take the user’s submission as an input, create a Question object and will send it to the Analyst

5 Functional Tests

5.1 Analyst Class Tests

Test	Description	Setup	Expected Result
ANL-1	Determine a valid Analyst Object	Pass parameters to create an Analyst object	no error should occur when creating the object
ANL-2	Return the name of the Analyst	Create an Analyst object with a name and use the Get-Name method	the name of the Analyst that we made
ANL-3	Pass a question to the Analyst	Create an analyst object and try to pass a question to it	the analyst should receive the question and be able to answer it

5.2 Graphics Class Tests

Class will consist solely of GUI building and testing will be done in person, so no explicit testing on the computer necessary.

5.3 Individual Stock Assessment Class Tests

Test	Description	Setup	Expected Result
ISACT-1	Pass no stock to be analyzed	pass no stock to the stock analyzer	NoStockError should be raised because there is nothing to analyze
ISACT-2	Pass a stock to be analyzed	Pass any random stock to be analyzed	Data pertaining to that stock and what the user should do with it
ISACT-3	Pass a non existing stock to be analyzed	Pass a stock that doesn't exist to the analysis method	a NonExistingStockError should be raised because unreal stock.

5.4 Owned Stock Class Tests

Test	Description	Setup	Expected Result
OSCT-1	user sends null values for creating OwnedStock Object	user attempts to add stock with null input fields	Throw InvalidOwnedStock error and prompt user to fill in input fields and try again
OSCT-2	OwnedStock does not exist	user attempts to access a OwnedStock that is not contained in their portfolio	Throw nullOwnedStock error and tell user that the stock requested does not exist in their portfolio

5.5 Portfolio Class Tests

Test	Description	Setup	Expected Result
PCT-1	invalid Owned-Stock object	The user attempts to add a invalid OwnedStock object to the portfolio	Throw InvalidOwnedStock error and prompt the user to re enter in stock information and try again
PCT-2	empty OwnedStock object	The user attempts to add an empty OwnedStock object to the portfolio	Throw EmptyOwnedStock error and prompt the user to fill out in stock information and try again
PCT-3	remove a stock from portfolio that does not exist	A user attempts to delete a stock that is not in their portfolio	Throw StockDoesNotExist error and tell user that the stock was not removed because it is not contained in the portfolio

5.6 Project Learning Tool Class Tests

Test	Description	Setup	Expected Result
PLTCT-1	Pass no stock to be helped learned	pass nothing to be helped learned	a NoStockError should be raised
PLTCT-2	Pass a non existent stock to be helped learned	Pass a non existent stock to be helped learned	a NonExistingStockError should be raised
PLTCT-3	Pass a stock to be helped learned	Pass a stock to the method	The program should show step by step how it is analyzed

5.7 Question Class Tests

Test	Description	Setup	Expected Result
QCT-1	Pass no question	User inputs nothing for a question	The program should not send through an empty question, prompt user for a question. Throw NoQuestionError.
QCT-2	Pass a question	User inputs a question for our expert to respond to	The program should send through the question to the expert's portal.

5.8 Stock Class Tests

Test	Description	Setup	Expected Result
SCT-1	Attempt to retrieve data for non-existent date	The user attempts to get data about a specific stock on a specific date that does not exist	Program returns message that this is an invalid date and throws InvalidDateError.
SCT-2	Attempt to retrieve data for existing date	The user retrieves data about a specific stock at an existing date	Program returns data about that stock at that date.
SCT-3	Attempt to retrieve data without internet connection	The user attempts to retrieve data about a specific stock at a date without an internet connection.	Program messages user that they need an internet connection in order to use this part of InvestMate. Throws FailedURLConnectionError.

5.9 Stock Analysis Class Tests

Test	Description	Setup	Expected Result
SACT-1	No stock is passed to be analyzed	User attempts to analyze a stock without inputting a stock	Throw NoAnalyzedStock error and prompt the user to input a stock to be analyzed.
SACT-2	Stock is passed to be analyzed	User provides a stock for the program to analyze	Program returns information about the stocks possible trends and viability based off of the stock's prior data

5.10 Stock Retrieval Class Tests

This class will consist of mainly API calls to grab up to date stock data from the internet and so testing will not be necessary

5.11 Stock Search Recommender Class Tests

Test	Description	Setup	Expected Result
SSRCT-1	No input fields are entered into the search	User doesn't provide information in filter fields for stock search	return the stock object of the stock that is the best to invest in.
SSRCT-2	The user selects analyst in stock search	The user attempts to search for the analysts stock of the week	return the stock object of the stock pick of the week variable
SSRCT-3	bad filter input	The user attempts to search for a stock with invalid input to filter fields	Throw BadFilterData error and prompt user to input valid filter information

5.12 User Class Tests

Test	Description	Setup	Expected Result
UCT-1	No user Information	User attempts to login without entering any user information	Throw EmptyUser error and prompt user to fill out input user fields
UCT-2	User does not exist	User attempts to log in with information that does not match any previous users	UserNotFound error and prompt user to either fill out input user fields again or create a new user

6 Defect Responsibility and Resolution

1. Issue Tracking

- (a) We plan on using Github's built-in issue tracking within our repository to track issues as it is easily accessible to all of our teammates and is a powerful enough tool to explain issues that need to be resolved.

2. Responsibilities

System Part	Developer	Responsibilities
Functional Testing and Investing Implementation	David Casente	Make sure testing has necessary coverage of code. Complete GUI functional tests. Handle backend material related to investing concepts and techniques.
Backend (API and Data Management)	Rembrandt van der Ploeg	Import stock information from the Alpha Vantage API. Process and handle data for consumption by the InvestMate. Handle user-side responsiveness through API usage optimization.
Front End (GUI/ backend connecting)	Mike O'Brien	Create an easy to use functional, GUI and make it aesthetically pleasing. Connect the front end to the back end. Feed data from the user to the back end

7 Exit Criteria

1. All J-Unit Tests have run and passed inspection.
2. All API related tests that cannot be checked through J-Unit testing have been thoroughly tested (manually), and all information taken from the API have been compared to trusted market data.
3. No stoppages throughout execution of the program, including those that could be caused by delayed API connections.